The Ant Colony Metaphor for Searching Continuous Design Spaces

G. BILCHEV[1] and I.C. PARMEE[2]

[1,2] *Plymouth Engineering Design Centre, University of Plymouth*
email: GBilchev@plymouth.ac.uk

Abstract

This paper describes a form of dynamical computational system—the ant colony—and presents an ant colony model for continuous space optimisation problems. The ant colony metaphor is applied to a real world heavily constrained engineering design problem. It is capable of accelerating the search process and finding acceptable solutions which otherwise could not be discovered by a GA. By integrating the Pareto optimality concept within the selection mechanism in GAs and Ant Colony it is possible to treat both hard and soft constraints. Hard constraints participate in a penalty term while soft constraints become part of a multi-criteria formulation of the problem.

Keywords: artificial ant colony, co-operative searches, dynamical computational systems, evolutionary computing, genetic algorithms

1. Introduction

A great majority of natural and artificial systems are of complex nature, and scientists choose more often than not to work on systems simplified to a minimum number of components in order to observe "pure" effects. An alternative approach, often known as the *complex systems dynamics* approach [G.Weisbuch], is to simplify as much as possible the components of the system, so as to take into account their large number. This idea has emerged from a recent trend in research known as the *physics of disordered systems.*

Complex dynamic systems in general show interesting and desirable behaviour as *flexibility* (in vision or speech understanding tasks the brain is able to cope with incorrect, ambiguous or distorted information, or even to deal with unforeseen or new situations without showing abrupt performance breakdown), *robustness* (keep functioning even when some parts are locally damaged), and they operate in a *massively parallel fashion.* Systems of this kind abound in nature. A vivid example is provided by the behaviour of a society of termites [P.J.Courtois].

> While individual termites are only able to perform very simple tasks such as transporting and dropping small quantities of earth in a quasi random fashion, an entire society of these insects is capable of building large nests with a sophisticated structure. Complex and organised behaviour thus emerges out of the massively parallel interactions between the many simple

members of the society. Moreover, this behaviour also shows flexibility and robustness. Flexibility, because the society is able to operate under very different circumstances, depending on the environment and on the structure of the subsoil. Robustness, because we can easily remove a number of termites without touching the society's nest building capabilities.

Complex dynamical systems show *emergent properties*. This means that the behaviour of the system as a whole can no longer be viewed as a simple superposition of the individual behaviours of its elements, but rather as a side effect of their collective behaviour. Contained in this notion is the idea that properties are not *a priori* predictable from the structure of the local interactions and that they are of functional significance.

Complex dynamical systems used for computation are called *dynamical computation systems*. The computation to be performed is contained in the dynamics of the system, which is determined by the nature of the local interactions between the many elements.

Many of the dynamical computation systems that have been developed today find their equivalent in nature. Examples include genetic algorithms [J.H.Holland], spin glass models [W.Kinzel], connectionist architectures [D.E.Rumelhart], reaction-diffusion systems [L.Steels] and simulated annealing [S.Kirkpatrick].

An important notion in dynamical computational systems is that of *interaction*. We concentrate on a particular kind of interaction: that of *co-operation*. Co-operation involves a collection of agents that interact by communicating information, or hints (usually concerning regions to avoid or likely to contain solutions) to each other while solving a problem. The information exchanged may be incorrect at times and should alter the behaviour of the agents receiving it. An example of co-operative problem solving is the use of the *genetic algorithm* to find states of high fitness in some space. In a genetic algorithm members of a population of states exchange pieces of themselves or mutate to create a new population, often containing states of higher fitness. Another example is *neural networks*, where the output of one neuron affects the behaviour of the neuron receiving it.

In this paper we concentrate on a particular type of computational task, that of *search*, which arises for problems with no known algorithmic method for direct solution construction.

Co-operative search methods are based on modifying individual search methods. A useful distinction is whether a method is *complete* or *incomplete*. Complete methods systematically examine states and are guaranteed to either eventually find a solution or terminate when no solution exists. By contrast, incomplete methods explore more opportunistically and may miss some states in the search space, hence they can never guarantee a solution does not exist. For parallel searches, a further issue is whether to

split the search space among the agents. In the simplest case, each agent examines the entire search space. However this can mean a single state is examined by more than one agent during the search. This can be avoided by partitioning the search space into disjoint parts and assigning one to each agent. In this partitioned search, agents only examine states in their assigned part of the space thus avoiding unnecessary duplicate examination of states. Restricting each agent to examine a state at most once, as well as partitioning the search space so that a state is not examined by more than one agent, may improve performance somewhat, but far less than the enhancement achieved by co-operation [S.H.Clearwater].

This paper describes a particular kind of dynamical computational system—that of the ant colony. We review results on modelling ant colonies for order based problems and then propose a model for continuous space optimisation.

2. The Ant Cycle Algorithm for order based problems

Problems like the Travelling Salesman Problem (TSP) and Bin-packing can be represented as a sequence of n items (n cities to be visited or n objects to be packed), where the actual order of the sequence determines a particular solution to the problem. Thus in general the search space consists of all $n!$ permutations. For such order based representations it is natural to apply the ant colony metaphor. In the following paragraphs the TSP [M. Dorigo] will be considered, because of its default interpretation of the items as cities, e.g. locations on a 2D map. Extensions to the Bin-packing problem and other order based representations will also be given.

For TSP with n cities the objective function is:

$$f(\pi) = \sum_{i=1}^{n-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(n)}, c_{\pi(1)})$$

where $d(c_i,c_j)$ is the distance between cities i and j, and $\pi(i)$ for $i=1,n$ defines a permutation. Let m be the number of ants. Then

$$m = \sum_{i=1}^{n} b_i(t)$$

where $b_i(t)$ is the number of ants in city i at time t.

There is also a global structure that represents the nest neighbourhood. In terms of the TSP it represents the distance between each pair of cities and the trail τ_{ij} left by the ants in the course of the algorithm execution. The trail is defined by:

$$_{ij}(t+n) = \rho \cdot \tau_{ij}(t) + \Delta\tau_{ij}(t,t+n)$$

where ρ is an evaporation constant, t is the beginning of a tour, and $t+n$ is the beginning of the next tour.

$$\Delta\tau_{ij}(t,t+n)=\sum_{k=1}^{m}\Delta\tau_{ij}^{k}(t,t+n)$$

$$\Delta\tau_{ij}^{k}(t,t+n)=\begin{cases}Q/L^{k} & \text{if } (i,j) \text{ is in the tour of ant } k\\ 0 & \text{otherwise}\end{cases}$$

In general Q/L^k is proportional to the success (fitness) of ant k, where Q is a constant and L^k is the tour length of the k-th ant.

Then during the next $(t+n)$ tour the probability to visit city j when being at city i is:

$$P_{ij}(t)=\begin{cases}\dfrac{[\tau_{ij}(t)]^{\alpha}\cdot[\eta_{ij}(t)]^{\beta}}{\sum_{k\in allowed}[\tau_{ik}(t)]^{\alpha}\cdot[\eta_{ik}(t)]^{\beta}} & \text{if } j\in allowed\\ 0 & \text{otherwise}\end{cases}$$

where *allowed* is a set of cities not visited for that particular tour, η_{ij} is local heuristics. For the TSP η_{ij} may be the greedy choice, e.g.

$$\min_{j} d(i,j)$$

Now the extension for the Bin-packing problem seems easy. Only the objective function is different, representing the nature of the problem. A good objective function is:

$$f(\pi)=B+\alpha/\mathrm{var}_{empty\_space}$$

where B is the number of boxes in the solution, $\mathrm{var}_{empty\_space}$ is the variance of the empty space of each box, and α is an appropriate coefficient representing a trade-off between the two criteria: $\mathrm{var}_{empty\_space}$ and B. In practice α gives total preference to B and $\mathrm{var}_{empty\_space}$ is used only to differentiate between two solutions with equal B's. If the local heuristics η_{ij} cannot be defined, then η_{ij} can be assumed to be equal to 1 for all ij.

The algorithm runs as follows: First P_{ij} 's are randomly initialised for some small values and m ants are allowed to make a tour. Then the solutions are compared and trail is laid proportionally to the ants' fitness. This alters the P_{ij} values so that on the next tour the probability of repeating (part of) previous good tours increases. This is reminiscent of *schemata propagation* in Genetic Algorithms where building blocks of high fitness pass from one agent to its offspring.

3. The Ant Colony Metaphor for Continuous Spaces

When the search space is continuous the ant cycle algorithm cannot be applied unless some kind of an order based representation is invented. Maintaining analogy with the foraging strategies of ant colonies we suggest a model applicable to continuous spaces. The main difficulty is how to model a continuous nest neighbourhood with a discrete structure. We have achieved this by representing a finite number of directions as vectors starting from a base point (nest). As we potentially have to cover all of the continuous space neighbourhood, these vectors are evolving in time according the ants' fitness (Fig.1).

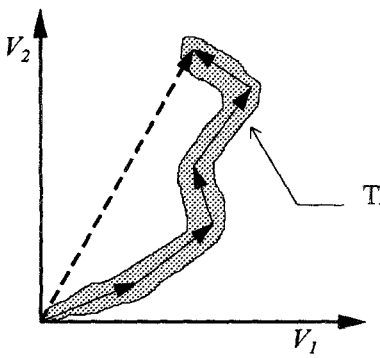


Fig. 1: A vector representing the actual path between the nest and the"food source" after five steps.

```
procedure Ant Colony Algorithm
begin
  t ← 0
  initialize A(t)
  evaluate A(t)
  while (not termination_condition) do
  begin
        t ← t + 1
        add_trail A(t)
        send_ants A(t)
        evaluate  A(t)
        evaporate A(t)
  end
end
```

Fig. 2: The structure of the Ant Colony Algorithm. `A(t)` is a data structure representing the nest and its neighbourhood.

The Ant Colony Algorithm

The structure of the Ant Colony Algorithm is shown in Fig. 2. Before the algorithm begins we have to determine the location of the nest. It should be a point in the search space which seems promising for fine local search exploitation. We suggest finding it by utilising a niching GA or a related strategy. Next we define a search radius R, which determines the extent of the subspace to be considered in each generation (cycle). Then `initialize A(t)` sends ants in various directions at a radius not greater than R (see Fig.3); `evaluate A(t)` is a call of the objective function for all ants; `add_trail A(t)` is proportionally (to the ants' fitness) adding trail quantity to the particular directions the ants have selected, `send_ants A(t)` sends ants by selecting directions using a Roulette wheel selection on the trail quantity and making a random step from the location of the best previous ant that have selected the same direction (see Fig.4), `evaporate A(t)` is decrementing the trail. The random step is implemented as

$$\Delta(t,R) = R \cdot (1 - r^{(1-t/T)^b})$$

where R is the search radius, r is a random number from [0..1], T is the maximal generation number, and b is a system parameter determining the degree of non-uniformity. $\Delta(t,R)$ returns a value in the range $[0..R]$ such that the probability of $\Delta(t,R)$ being close to 0 increases as t increases. R is determined by the extent of the search subspace we want to cover during the run.

If certain directions do not result in improvement, they do not participate in the trail adding process and the reverse (evaporation) process diverts attention away from them. This can be thought of as an analogy of a food source exhausting.

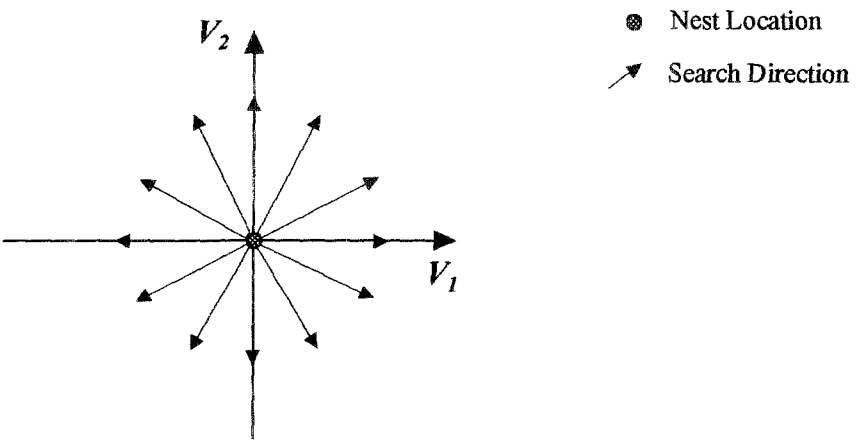
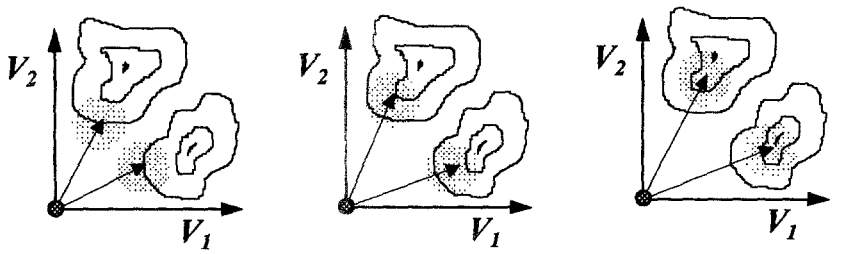
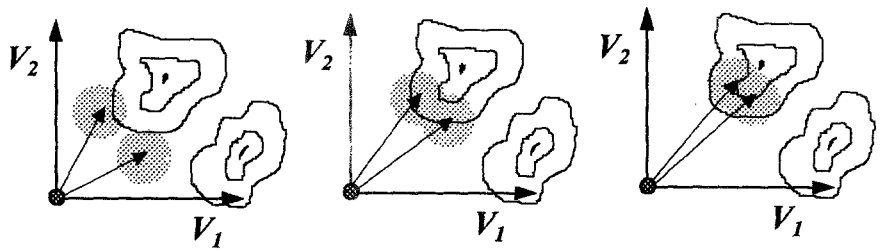


Fig. 3: Two dimensional Nest neighbourhood model with twelve search directions. Each direction evolves in time according to the fitness of the ants that have selected it. Typical direction evolution is shown in Fig. 4.



a) different basins of attraction for the two directions



b) the same basin of attraction for the two directions

Fig 4: Evolution of directions. The shaded region shows the search radius R at each step.

To make the model more accurate a random walk and trail diffusion can be added. The basic trail diffusion idea is shown on fig. 5. It is important to notice the analogy between trail diffusion in the Ant Colony and arithmetic crossover in GAs. In arithmetic crossover the new chromosome is constructed from its parents by a linear combination of the individual parameters: $\alpha_i x_i^1 + (1-\alpha_i) x_i^2$. The distribution of α_i can be assumed normal with mean value equal to $0.5(x_i^1 + x_i^2)$ and the probability of generating new offspring with particular parameter values can be represented as contour plot in the form of concentrated hyperspheres, i.e., closer to the centre, greater the probability (fig. 5a). In the Ant Colony the intersection of the diffused trail from two different paths form a virtual path with a location probability determined by the superposition of the diffused trail strength (fig. 5b).

An obvious limitation of the current algorithm is the lack of a model for the exhausting of the food source. Thus it is possible for search agents to repeat already tracked and assumed exhausted paths.

4. Experimental Results

The Ant Colony model is used in a real world heavily constrained engineering design problem. The domain involves preliminary air-frame design and the definition of a flight trajectory for an air-launched winged rocket that will achieve orbit before

returning to atmosphere for a conventional landing. The problem is extremely sensitive to five non-explicit (embodied in a simulation program) non-linear constraints relating to air-speed and climb angle. Constraints also affect the physical parameters describing the air-frame and engine configuration. The problem is to minimise the empty weight of the vehicle through a space of seven continuous and one discrete design variables, subject to these five non-linear constraints. Initial discussion revealed a degree of doubt as to whether a feasible solution to the problem actually exists and preliminary experimentation using a GA with the constraint violation as a fitness function could only achieve solution exhibiting minimum constraint violation.

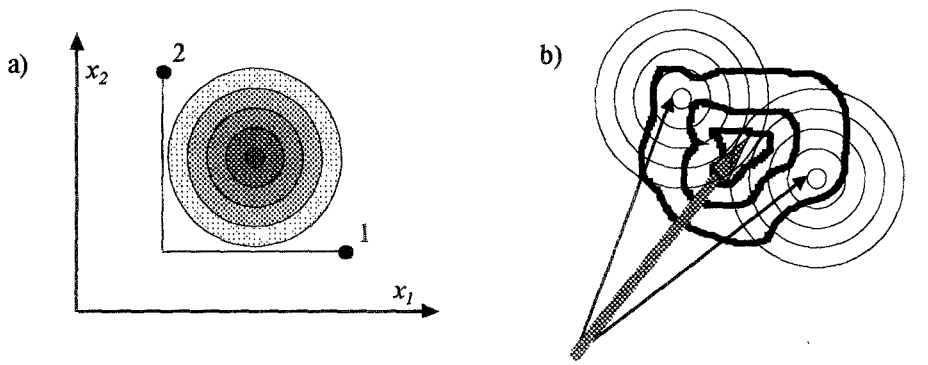


Fig. 5: (a) Contour plot of the probability for generating new offspring from two parents (1 and 2) by arithmetic crossover in GAs, (b) Superposition of trail diffusion (the concentrated circles) forms a kind of virtual path (the grey vector). The graph is superimposed over the contour plot of the fitness function to show the expected effect of trail diffusion.

A GA with floating point representation is used because it offers (1) a significant reduction in the length of the chromosome, (2) an ability to express and operate on parameters in their natural base 10 encoding, and (3) ease in interfacing to other algorithms (e.g. standard numerical optimisation techniques, regression analysis, etc.). A rank based selection scheme with linear normalisation is utilised.

It is evident that a secondary search process is required. As the fitness landscape is still very complex and detailed even when considering small neighbourhoods (0.01% of the parameters range) we cannot use a hill climber search as it will get stuck in local optima. The Ant Colony was initially designed for fine local search applied after the GA has found promising clusters for future exploration, although some preliminary results show it can also search well in larger spaces.

A particular run of the GA is shown in Fig. 6. Fig. 7 shows the result obtained when applying the Ant Colony from the point found by the GA in Fig. 6.

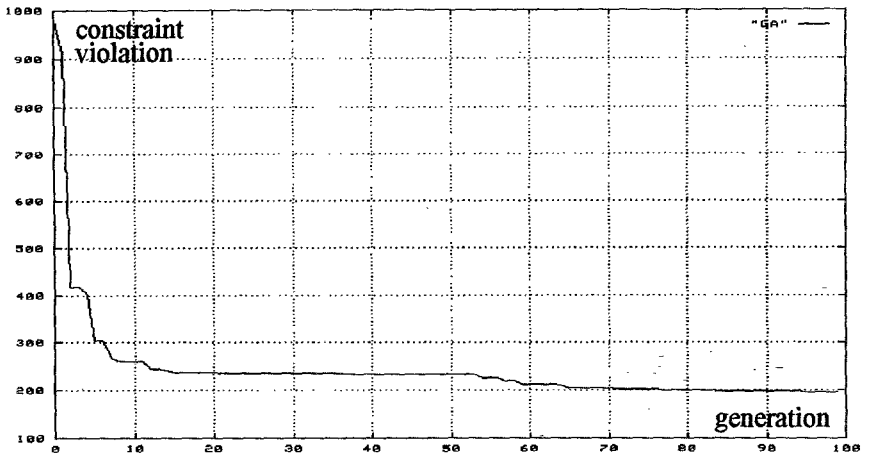


Fig. 6: A particular run of a GA for 100 generations. The graph shows the distance from the constraint values defined as the sum of the squares of the difference between each current value of the particular constrained variable and its constrained value.

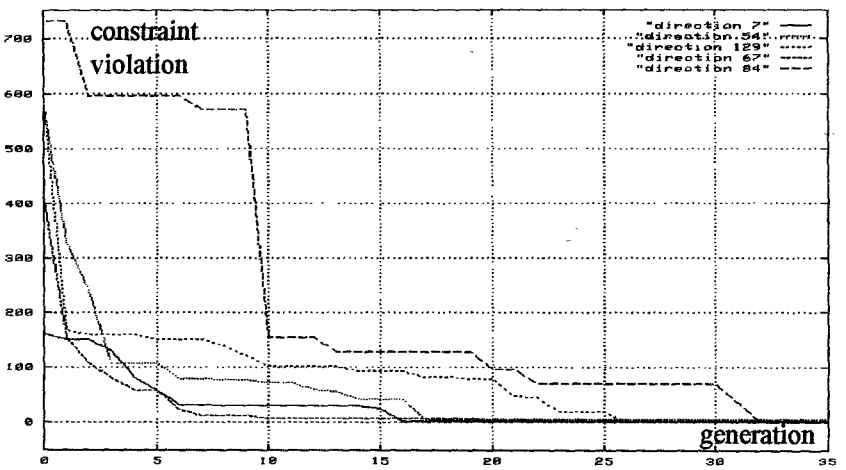


Fig. 7: The Ant Colony Algorithm applied on the point found by the GA from Fig. 6. The evolution of the distance from the constrained values of five directions is shown.

Current work at the **P**lymouth **E**ngineering **D**esign **C**entre [G. Bilchev, and I.C. Parmee] involves the integration of the Pareto optimality concept into the selection mechanism of GAs and Ant Colony. The main motivation is that standard non-linear programming methods cannot treat both soft and hard constraints. Previous work within the Centre has addressed the representation of soft constraints as multi-

objectives by utilising a modified VEGA approach [I.C. Parmee, and G. Purchase]. A Pareto approach will also allow soft constraints to be incorporated as multi-criteria while hard constraints can be treated in the usual way in penalty terms.

5. Sensitivity Analysis

A sensitivity measure is defined in terms of maximum and average risk to achieve degraded real design when deviating from the numerically represented design solution. That risk is unavoidable because of the physical incapability to achieve the exact values of the design variables. It may happen that even when using numerically stable algorithms the found optimal design solution lies within a very sensitive region and a small perturbation in the design variables can lead to an enormous change in the overall design solution [I.C. Parmee and M.J. Denham]. This is a property of the problem itself and does not depend on the actual optimisation algorithm. So when making decisions the engineering designer, among others, should take into consideration the sensitivity of a given solution.

A generic method for calculating the sensitivity is presented, which is applicable to problems with non-explicit objective and constraint functions. The proposed sensitivity measure is also capable of representing design variables interaction. The price paid for that is increase in the number of the objective function calls. A method for incorporating sensitivity analysis with the search process and thus essentially reducing the number of the objective function calls is proposed.

5.1 Definition

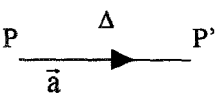
Let's denote our objective function by $F:\Re^n \rightarrow \Re$ or $F(x_1, x_2, ..., x_n)$. Then F can be considered a scalar field and the gradient is defined by:

$$\text{grad } F = \frac{\partial F}{\partial x_1}\cdot\vec{i}_1 + \frac{\partial F}{\partial x_2}\cdot\vec{i}_2 + \cdots + \frac{\partial F}{\partial x_n}\cdot\vec{i}_n$$

where $\vec{i}_1,\ldots,\vec{i}_n$ are the unit vectors of an orthogonal co-ordinate system. The gradient at each point has a length and direction that is independent of the particular choice of Cartesian co-ordinates. If at a point **P** the gradient of F is not the zero vector, it has the direction of the maximum increase of F at **P**.

The directional derivative is defined as:

$$D_a F = \frac{dF}{d\Delta} = \lim_{\Delta\rightarrow 0}\frac{F(P') - F(P)}{\Delta}$$



$P = (x_1,...,x_n)$
$P' = (x_1+\Delta x_1,...,x_n+\Delta x_n)$

and is effectively calculated by:

$$D_a F = \frac{1}{|\vec{a}|}\ \vec{a} \bullet \text{grad}\ F = |\text{grad}\ F| \cdot \cos\alpha$$

where α is the angle between $\vec{a}$ and grad F at a particular point.

Using finite differences the directional derivative can be approximated by:

$$D_a F = \frac{dF}{d\Delta} \approx \frac{F(P') - F(P)}{\Delta}$$

for some small positive real value Δ.

If the gradient has the direction of the maximum increase of F at **P** then:

$$\vec{G}(x_1^P, \ldots, x_1^P) = \text{grad}\ F = \lim_{\Delta \to 0} \frac{(\vec{P}' - \vec{P})}{|\vec{P}' - \vec{P}|} \cdot \max_{\substack{\forall P' \\ (x_1^{P'} - x_1^P)^2 + \cdots + (x_n^{P'} - x_n^P)^2 = \Delta^2}} \frac{F(P') - F(P)}{\Delta}$$

Now a *neighbourhood sensitivity* can be defined as:

$$\vec{\hat{S}}_{\max}(\delta) = \vec{\hat{G}}(x_1^P, \ldots, x_n^P) = \frac{(\vec{P}' - \vec{P})}{|\vec{P}' - \vec{P}|} \cdot \max_{\substack{\forall P' \\ (x_1^{P'} - x_1^P)^2 + \cdots + (x_n^{P'} - x_n^P)^2 = \delta^2}} |F(P') - F(P)|$$

$\vec{\hat{S}}$ is an estimation of the maximum degradation that can be achieved in the real design. In a similar manner an estimation of the average degradation is defined:

$$\vec{\hat{S}}_{\text{mean}}(\delta) = \vec{\hat{G}}(x_1^P, \ldots, x_n^P) = \frac{(\vec{P}' - \vec{P})}{|\vec{P}' - \vec{P}|} \cdot \text{mean}_{\substack{\forall P' \\ (x_1^{P'} - x_1^P)^2 + \cdots + (x_n^{P'} - x_n^P)^2 = \delta^2}} |F(P') - F(P)|$$

The neighbourhood sensitivity is a vector such that its magnitude represents the difference of the fitness function calculated at two points (**P'** and **P**) while its direction gives information about the actual design variables influence on that fitness function change.

5.2 Calculating Sensitivity

The definitions of $\vec{\hat{S}}_{\max}$ and $\vec{\hat{S}}_{\text{mean}}$ imply the algorithms for their calculation. Finding $\vec{\hat{S}}_{\max}$ is a search process on the surface of a hypersphere defined by

$(x_1^{P'} - x_1^P)^2 + \cdots + (x_n^{P'} - x_n^P)^2 = \delta^2$ while finding $\vec{\hat{S}}_{mean}$ is sampling that hypersphere and calculating the average |F(P')–F(P)|.

The Ant Colony algorithm is adopted to find $\vec{\hat{S}}_{max}$ because it is a robust multi-modal search technique that will give information about various risky directions of degradation. Moreover the intention is to later avoid a separate search for calculating the sensitivity by integrating it with the second phase of the overall design solution search process. This can be implemented by keeping track of all model evaluations during that second phase Ant Colony search and then after a solution is accepted the sensitivity can be calculated by referencing the previous model call results. It may happen that in certain directions the hypersphere is not well sampled which will require some extra model calls. It is recommended that the sensitivity is now calculated through a hypersphere layer defined by:

$$(\delta - \varepsilon)^2 \le (x_1^{P'} - x_1^P)^2 + \cdots + (x_n^{P'} - x_n^P)^2 \le (\delta + \varepsilon)^2$$

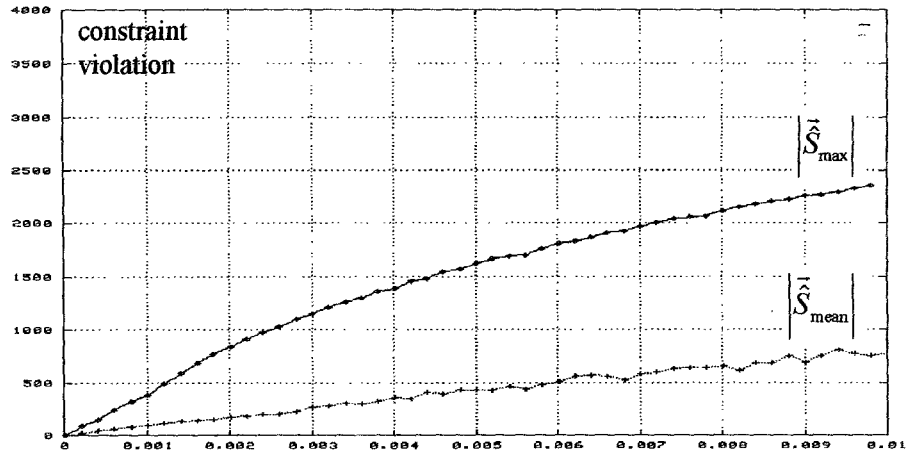
where ε defines the thickness of the layer. The graph now will be more like a histogram.

5.3 Experimental Results

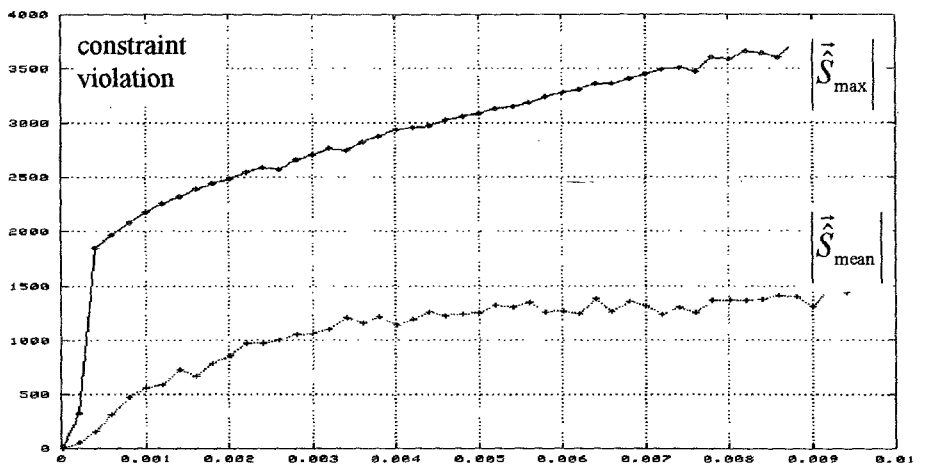
The sensitivity is calculated at two points found by the search procedure described in section 4. Considering that both points seem to be promising design solutions according to the constraint and/or multi-criteria satisfaction the sensitivity can be used as an additional decision criterion. Fig. 8 clearly shows that the solution at points 1 is less sensitive than the solution at point 2. The engineering designer may also want to take into consideration the degree to which each design variable contributes to the sensitivity. Table 1 shows that information for point 1.

δ%	Δα	Δγ	...	Δh_{MECO}
0.0001	-0.51527	0.50107	...	-0.15359
0.0002	-0.48958	0.47331	...	-0.01903
0.0003	0.56719	-0.41718	...	-0.02065
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...
0.0099	-0.38798	0.33810	...	-0.06375
0.0100	-0.28638	0.25271	...	-0.07806

Table 1: Direction information of the maximum sensitivity for point 1 (fig. 8). The numbers in the table define a direction of a unit radius vector in a hyperspace.



point 1



point 2

Fig. 8: Sensitivity calculated at two points of the search space

6. Discussion

The Ant Colony dynamics provide a control for the trade-off between exploration and exploitation during a search process. In the present paper we have used it with a simple hill climbing algorithm in order to show the power of co-operation, but it can be integrated in a similar way with many search techniques. We have found several advantages in the Ant Colony Metaphor used for finding good feasible solutions:

initial direction information to be lost due to migration of agents from one direction to another (see Fig. 4b), but this can be controlled by parameters of the Ant Colony dynamics.)

- Many search directions are considered in parallel
- Easy to integrate with many search techniques (hill climbers, GA's, etc.)
- Better than local search if the search space contains numerous basins of attraction. (We also believe it is better than local search for long path problems [J.Horn, D.Goldberg].)

We have become more convinced that rather than spending all the effort in developing a monolithic program or perfect heuristic, it may be better to have a set of relatively simple co-operating processes working concurrently on the problem while communicating their partial results.

Acknowledgements
This research is supported by the Plymouth Engineering Design Centre at the University of Plymouth. British Aerospace has provided experimental design software and guidance. We thank these organisations for their continuing support.

References

D.E.Rumelhart and J.L.McClelland, Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1, Bradford Books, Cambridge, MA

Deniz Yuret, and Michael de la Maza, Dynamic Hill Climbing: Overcoming the limitations of optimization techniques, Technical report, Numinous Noetics Group, Artificial Intelligence Laboratory, MIT, MA 02139

George Bilchev, Evolutionary Algorithms for the Bin-packing Problem, Chapter 4, Comparison between Ant Colony Search and Genetic Algorithms, *MSc thesis*, 1994, New Bulgarian University, Sofia 1125, Bulgaria (in Bulgarian)

George Bilchev, and I.C. Parmee, Searching Heavily Constrained Design Spaces, *Procs. of the 22nd International Conference on CAD-95*, 8-13 May, 1995, Yalta, Ukraine.

George Bilchev, and I.C. Parmee, Natural Self-organizing Systems, Internal Report PEDC-03-95, Engineering Design Centre, University of Plymouth, UK

Gerard Weisbuch, Complex Systems Dynamics, Lecture Notes Volume II, Santa Fe Institute, Studies in the Sciences of Complexity, Addison-Wesley, 1991

I.C.Parmee, and G.Purchase, The Development of a Directed Genetic Search Technique for Heavily Constrained Design Spaces, in *Proc. of Adaptive Computing in Engineering Design and Control*, Plymouth, 1994.

I.C. Parmee and M.J. Denham, Emergent Computing Methods in Engineering Design, NATO Advanced Research Workshop, Nafplio, Greece, August 1994.

J.H.Holland, Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications in Biology, Control, and Artificial Intelligence, University of Michigan Press, Ann Arbor, 1975

J.Torreele, Optimization by Simulated Annealing. Introduction and Case Study, AI-MEMO 88-18, AI-lab VUB, Brussles

Jeffray Horn, N. Nafpliotis, and D.E.Goldberg, A Niched Pareto Genetic Algorithm for multiobjective optimization, in *Proceedings of the First IEEE Conference on Evolutionary Computation, IEEE World Congress on Computational Intelligence, Volume 1,* 1994, Piscataway, NJ

Jeffrey Horn, David E. Goldberg, and Kalyanmoy Deb, Long Path Problems, *Proceedings of Parallel Problem Solving from Nature*, Lecture Notes in Computer Science, Springer-Verlag, 1995

L.Nadel and D.Stein, eds., Better than the Best: The Power of Cooperation, *Complex Systems*, 163-184, Addison-Wesley 1993

L.Steels, Artificial Intelligence and Complex Systems, AI-MEMO 88-2, AI-lab VUB, Brussels

Marco Dorigo, The Ant Cycle Algorithm, Technical report, Free University of Brussels

P.J.Courtois. On line and Space Decomposition of Complex Structures, *Comm. of the ACM,* Vol.28, no.6

S.Kirkpatrick, Gelatt C.D., M.P.Vechi, Optimisation by Simulated Annealing, *Science*, Vol.220, No.4598, May, 1983

Scott H. Clearwater, Bernardo A. Huberman, and Tad Hogg, Cooperative Problem Solving, in B. Huberman, editor, *Computation: The Micro and the Macro View*, pages 33-70, World Scientific, Singapore, 1992

W.Kinzel, Learning and Pattern Recognition in Spin Glass Models, 1985